МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) «Компьютерные науки и прикладная математика»
Образовательный центр Института № 8
Группа М8О-411Б-22 **Направление подготовки** 02.03.02
Фундаментальная информатика и информационные технологии
Профиль Информатика и компьютерные науки
Квалификация бакалавр

УТВЕРЖДАЮ

Директор Дирекции Института № 8 _____ _____
(подпись) (инициалы, фамилия)
10 февраля 2026г.

ЗАДАНИЕ
на выпускную квалификационную работу бакалавра

Обучающийся Заславцев Максим Владимирович
(фамилия, имя, отчество полностью)
Руководитель Романенков Александр Михайлович
(фамилия, имя, отчество полностью
Кандидат технических наук, доцент, доцент образовательного центра Института №8 “Компьютерные науки и прикладная математика”, Образовательный центр Института №8 “Компьютерные науки и прикладная математика”
(ученая степень, ученое звание, должность и место работы)

1. Наименование темы Разработка системы долговременного защищенного хранения цифровых активов с применением гибридных постквантовых криптографических алгоритмов на платформе Java 25

2. Срок сдачи обучающимся законченной работы 11 мая 2026 г.

3. Задание и исходные данные к работе
Разработать на платформе Java 25 программный модуль для долговременной квантово-устойчивой защиты цифровых активов на основе гибридных алгоритмов шифрования (X25519 и ML-KEM), гибридной электронной подписи (Ed25519 и ML-DSA) и алгоритмов предварительного сжатия данных.

Перечень иллюстративно-графических материалов:

№ п/п	Наименование	Количество листов
1	**Раздаточный материал**	**15**

4. Перечень подлежащих разработке разделов и этапы выполнения работы

п/п	Наименование раздела или этапа	Трудоёмкость в % от полной трудоёмкости ВКРБ	Срок выполнения	Примечание
1	Анализ предметной области, обзор гибридных постквантовых криптографических алгоритмов и формулировка требований к системе.	15	10.02.2026	
2	Проектирование архитектуры программного модуля долговременного хранения и математическое обоснование механизма гибридного конвейера (KEM + HKDF).	15	25.02.2026	
3	Программная реализация подсистемы в виде Spring Boot Starter на Java 25, включая разработку методов конвертного шифрования и спектро-энтропийного алгоритма сжатия.	30	28.03.2026	
4	Комплексное тестирование, профилирование производительности модуля и проведение экспериментальной оценки.	35	29.04.2026	
5	Оформление ВКР и итогового отчета.	5	10.05.2026	

5. Исходные материалы и пособия

1. FIPS PUB 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. – National Institute of Standards and Technology (NIST), 2024.
2. H. Krawczyk, P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF): RFC. – 2010. – № 5869.

6. Дата выдачи задания 10 февраля 2026

Руководитель _____
(подпись)

Задание принял к исполнению _____
(подпись)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

О Т З Ы В
РУКОВОДИТЕЛЯ

Обучающийся Заславцев Максим Владимирович
(фамилия, имя, отчество полностью)

Институт (Филиал) «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-411Б-22 **Направление подготовки** 02.03.02 Фундаментальная информатика и информационные технологии

Профиль Информатика и компьютерные науки

Квалификация бакалавр

Наименование темы Разработка системы долговременного защищенного хранения цифровых активов с применением гибридных постквантовых криптографических алгоритмов на платформе Java 25

Руководитель Романенков Александр Михайлович, Кандидат технических наук, доцент, доцент образовательного центра Института №8 “Компьютерные науки и прикладная математика”, Образовательный центр Института №8 “Компьютерные науки и прикладная математика”
(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Заславцев Максим Владимирович успешно выполнил выпускную квалификационную работу на тему «Разработка системы долговременного защищённого хранения цифровых активов с использованием гибридных постквантовых криптографических алгоритмов на платформе Java 25». В работе проведён глубокий анализ предметной области: рассмотрена угроза отложенной расшифровки, изучены постквантовые стандарты NIST и ограничения JVM в части утечек ключевой информации через дампы памяти. Разработана квантово-устойчивая подсистема хранения в виде Spring Boot Starter, реализующая принцип криптоагильности. Автор спроектировал KDF-комбайнер на базе HKDF для объединения классической и постквантовой энтропии, реализовал гибридную электронную подпись, а также механизмы защиты памяти и от тайминг-атак. Научную ценность представляет предложенный спектро-энтропийный алгоритм сжатия на основе быстрого преобразования Фурье для обработки телеметрических данных. Проведено профилирование системы, позволившее сократить время выполнения криптографических операций на 29% за счёт оптимизированного конвейера «Compress-then-Encrypt» при сохранении высокой пропускной способности. В ходе работы Заславцев М.В. продемонстрировал высокий уровень самостоятельности, ответственности и инженерной подготовки, а также способность эффективно решать задачи на стыке прикладной математики и программной инженерии.

Работа проверена на объем заимствования. % заимствования – 7,29%

Заключение: считаю, что работа заслуживает оценки «отлично», а автор – Заславцев Максим Владимирович – присвоения квалификации «бакалавр» по направлению подготовки «Фундаментальная информатика и информационные технологии».

10 мая 2026г. Руководитель
(подпись)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) «Компьютерные науки и прикладная математика»
Образовательный центр Института № 8
Группа М8О-411Б-22 **Направление подготовки** 02.03.02
Фундаментальная информатика и информационные технологии
Профиль Информатика и компьютерные науки
Квалификация бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Разработка системы долговременного защищенного хранения цифровых активов с применением гибридных постквантовых криптографических алгоритмов на платформе Java 25

Автор ВКРБ Заславцев Максим Владимирович (_____)
(фамилия, имя, отчество полностью)
Руководитель Романенков Александр Михайлович (_____)
(фамилия, имя, отчество полностью)

К защите допустить

И.о. директора Образовательного
центра Института № 8 Крылов С.С. (_____)
(фамилия, имя, отчество полностью)

12 мая 2026г.

Москва 2026

РУКОНТЕКСТ
text.rucont.ru

Справка о проверке на наличие заимствований

Имя файла: diploma.pdf
Автор: Заславцев Максим Владимирович
Заглавие: Разработка системы долговременного защищенного хранения цифровых активов с применением гибридных постквантовых криптографических алгоритмов на платформе Java 25
Год публикации: 2026
Комментарий: *Не указан*
Коллекции: Интернет 2.0, Научные статьи 2.0, Википедия, Англоязычная Википедия, Коллекция Энциклопедий, Библиотека Либрусек, Университетская библиотека, Коллекция КФУ, ВКР Российского университета кооперации, Коллекция АПУ ФСИН, Коллекция ПГУТИ, Репозиторий открытого доступа СПб гос. ун-та, Научная электронная библиотека "КиберЛенинка", ЦНМБ Сеченова, Авторефераты ВАК, Диссертации ВАК, Диссертации РГБ, Авторефераты РГБ, Готовые рефераты, ФИПС. Изобретения, ФИПС. Полезные модели, ФИПС. Промышленные образцы, Коллекция Руконт, Библиотека им. Ушинского, Готовые рефераты (часть 2), Открытые научные источники, НЭБ, БиблиоРоссика, Правовые документы I, Правовые документы II, Правовые документы III, Собрание законодательства Российской Федерации

Результат проверки

Оценка оригинальности документа: 89%

Оригинальные фрагменты: 88,87%
Обнаруженные заимствования: 7,29%
Цитирование: 3,84%



Работу проверил: Романенков Александр Михайлович

Дата:

Подпись:

© ООО НЦР «Руконт», 2026

РЕФЕРАТ

Выпускная квалификационная работа состоит из 51 страницы, 9 рисунков, 4 таблиц, 20 использованных источников, 1 приложения.

ПОСТКВАНТОВАЯ КРИПТОГРАФИЯ, ГИБРИДНАЯ КРИПТОГРАФИЯ, ML-KEM, ML-DSA, SPRING BOOT STARTER, КРИПТОАГИЛЬНОСТЬ, HARVEST NOW DECRYPT LATER, КОНВЕРТНОЕ ШИФРОВАНИЕ, HKDF, JAVA 25, COMPRESS-THEN-ENCRYPT, AES-GCM

Объект разработки: программная криптографическая подсистема долговременного защищённого хранения цифровых активов, включающая механизмы гибридного шифрования, управления ключами и постквантовой электронной подписи.

Предмет исследования: методы интеграции постквантовых стандартов NIST (FIPS 203, FIPS 204) и гибридных криптографических схем в экосистему Spring Boot для обеспечения конфиденциальности данных на горизонте 10+ лет.

Цель работы

Целью выпускной квалификационной работы является разработка повторно используемого программного модуля (Spring Boot Starter) для долговременного защищённого хранения цифровых активов. Модуль должен обеспечивать квантово-устойчивую криптографическую защиту за счёт применения гибридных схем (классические + постквантовые алгоритмы) и поддерживать принцип криптоагильности — возможность декларативной замены алгоритмов без переработки прикладного кода бизнес-логики.

Задачи работы:

1) Проанализировать угрозу Harvest Now, Decrypt Later (HNDL) и влияние квантовых вычислений на стойкость классических криптосистем.

2) Спроектировать архитектуру Spring Boot Starter, поддерживающую автоконфигурацию и принцип crypto-agility.

3) Реализовать ядро криптографической подсистемы: гибридный механизм инкапсуляции ключа (KEM), конвертное шифрование данных и формирование постквантовой цифровой подписи (ML-DSA).

4) Интегрировать механизмы управления ключевой информацией и оптимизации хранения (предварительное сжатие данных).

5) Провести экспериментальную оценку производительности и профилирование разработанного решения.

Результаты работы: Разработан, протестирован и готов к внедрению программный модуль, обеспечивающий полный жизненный цикл безопасного хранения файла (сжатие → гибридная генерация ключа → конвертное шифрование → постквантовая подпись) с унифицированным API.

Новизна результатов:

1) Практическая интеграция постквантовых стандартов FIPS 203/204 в прикладной сервис на платформе Java 25 с использованием механизма HKDF для безопасного объединения ключей.

2) Разработка математически обоснованного конвейера «Compress-then-Encrypt», компенсирующего увеличенный размер постквантовых криптографических артефактов.

3) Формирование архитектурного шаблона защищённого хранилища, ориентированного на долговременную конфиденциальность и противодействие угрозе HNDL.

Область применения: Информационные системы аэрокосмической и оборонной отраслей: системы управления жизненным циклом изделий (PLM), защищенные архивы конструкторской и технологической документации, хранилища полетных данных и телеметрической информации космических аппаратов. Применение модуля критически актуально для предприятий авиастроения, где срок эксплуатации техники значительно превышает горизонт появления квантовых компьютеров, что требует защиты интеллектуальной собственности и данных БПЛА от угрозы HNDL.

Прогноз развития объекта исследования: Ожидается рост практического внедрения постквантовой криптографии в прикладных системах. Прогнозируется появление отечественных сертифицируемых PQC-реализаций, что позволит бесшовно перевести разработанный модуль с алгоритмов NIST на будущие стандарты ГОСТ благодаря заложенной архитектуре криптоагильности.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 5

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 8

ВВЕДЕНИЕ 11

1 Анализ алгоритмов постквантовой криптографии и принципов построения гибридных схем 13
1.1 Алгоритм Шора и угроза для классических асимметричных систем 13
1.2 Краткий обзор стандартов постквантовой криптографии NIST 14
1.3 Архитектура и математическое обоснование гибридных криптографических схем 16

2 Обзор существующих решений 19
2.1 Bouncy Castle и реализация PQC в экосистеме Java 19
2.2 Проект Open Quantum Safe (libOQS) и нативные Java-обёртки 19
2.3 Коммерческие решения: PQShield PQCryptoLib-SDK 19
2.4 Обоснование разработки собственного Spring Boot Starter . . . 20
2.5 Платформа Java 25 как технологическая база 20

3 Проектирование и реализация программного модуля 22
3.1 Архитектура решения и модульная структура 22
3.2 Реализация гибридной инкапсуляции ключей и механизма KDF 22
3.3 Конвертное шифрование и формат данных (Data-at-Rest) . . . 24
3.4 Обобщенный конвейер криптографических преобразований . 25
3.5 Безопасность памяти и предотвращение атак по побочным каналам 27
3.6 Автоконфигурация Spring Boot и реализация криптоагильности 28

4 Интеграция механизмов сжатия данных для оптимизации PQC-артефактов 31
4.1 Обоснование необходимости сжатия 31
4.2 Интеграция стандартных алгоритмов компрессии 32
4.3 Экспериментальный метод спектро-энтропийного сжатия (FOURIER) 33

5 Экспериментальная оценка и профилирование программного модуля 36
5.1 Оценка криптографических накладных расходов (Overhead) . 36
5.2 Сравнительный анализ алгоритмов сжатия 36
5.3 Временные метрики и парадокс производительности 39
5.4 Эвристический адаптивный обход (Adaptive Fallback) 39
5.5 Профилирование генерации ключей и электронной подписи (ML-DSA) 40
5.6 Оценка масштабируемости уровней безопасности NIST 40
5.7 Пропускная способность системы (Throughput) 41
5.8 Профилирование подсистемы электронной подписи (ML-DSA vs Классика) 42

ЗАКЛЮЧЕНИЕ 45

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 47

ПРИЛОЖЕНИЕ А 50

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей итоговой аттестационной работе применяют следующие термины с соответствующими определениями:

Алгоритм Шора — квантовый алгоритм, позволяющий решать задачи факторизации целых чисел и дискретного логарифмирования за полиномиальное время, что делает уязвимыми классические асимметричные криптосистемы (RSA, ECC)

Атака по побочным каналам (Side-channel attack) — класс криптоаналитических атак, использующих информацию о физическом процессе работы алгоритма или среды исполнения (например, время выполнения операций, электромагнитное излучение, дампы оперативной памяти)

Гибридная криптографическая схема — архитектура криптографической защиты, в которой совместно и одновременно используются классические и постквантовые алгоритмы для повышения стойкости и сохранения обратной совместимости

Гибридный KEM — механизм установления общего секрета, объединяющий классический алгоритм согласования ключа (например, X25519) и постквантовый механизм инкапсуляции с использованием функции деривации ключа (KDF)

Данные в состоянии покоя (Data-at-Rest) — цифровые данные, физически сохраненные на цифровых носителях (в базах данных, файловых системах, объектных хранилищах), в отличие от данных, находящихся в процессе передачи по сети (Data-in-Transit)

Долговременная конфиденциальность — свойство информационной системы обеспечивать сохранение секретности активов на длительном временном горизонте (10 и более лет), в том числе при скачкообразном росте вычислительных возможностей нарушителя

Конвертное шифрование (Envelope Encryption) — подход к защите информации, при котором полезные данные шифруются симметричным ключом

(DEK — Data Encryption Key), а сам этот ключ дополнительно защищается асимметричным или гибридным ключом (KEK — Key Encryption Key)

Криптоагильность (Crypto-agility) — архитектурный принцип проектирования программного обеспечения, предусматривающий возможность декларативной замены криптографических алгоритмов и параметров без переработки бизнес-логики прикладной системы

Криптографически релевантный квантовый компьютер (CRQC) — гипотетическая квантовая вычислительная система, обладающая достаточным количеством логических кубитов и механизмами коррекции ошибок для успешного применения алгоритма Шора против современных стандартов криптографии

Механизм инкапсуляции ключа (Key-Encapsulation Mechanism, KEM) — криптографический примитив, позволяющий двум сторонам безопасно согласовать симметричный секретный ключ по открытому каналу связи

Обучение с ошибками (Learning With Errors, LWE / Module-LWE) — фундаментальная математическая задача в решёточной криптографии, стойкость которой базируется на сложности восстановления секретного вектора по системе линейных уравнений с добавленным случайным шумом

Постквантовая криптография (PQC) — раздел криптографии, изучающий и стандартизирующий алгоритмы, устойчивые к криптоанализу как на классических, так и на квантовых вычислительных системах

Решёточная криптография (Lattice-based cryptography) — ведущее направление постквантовой криптографии, стойкость алгоритмов которого основана на вычислительной сложности геометрических задач в многомерных решётках (SVP, CVP)

Сжатие перед шифрованием (Compress-then-Encrypt) — архитектурный конвейер обработки данных, при котором алгоритмы компрессии применяются к избыточному открытому тексту до его криптографического преобразования, так как качественный шифротекст обладает максимальной энтропией

и не подлежит сжатию

Функция формирования ключа (Key Derivation Function, KDF) — криптографический алгоритм для безопасного вывода одного или нескольких секретных ключей из базового ключевого материала. В рамках работы применяется HKDF (HMAC-based Extract-and-Expand KDF)

Цифровой актив — электронный объект хранения (документ, архив, резервная копия), обладающий ценностью для владельца и требующий обеспечения целостности, подлинности и контролируемого доступа

Энтропия (информационная) — мера неопределенности данных. В контексте работы учитывается при проектировании конвейера гибридного смешивания ключей и спектро-энтропийного сжатия активов

Harvest Now, Decrypt Later (HNDL) — модель угроз, при которой злоумышленник перехватывает и сохраняет зашифрованные криптоконтейнеры в настоящем с целью их расшифрования в будущем после появления CRQC

ML-DSA — стандартизованный NIST в FIPS 204 постквантовый алгоритм цифровой подписи на основе модульных решёток

ML-KEM — стандартизованный NIST в FIPS 203 постквантовый механизм инкапсуляции ключа на основе модульных решёток

Spring Boot Starter — переиспользуемый программный модуль экосистемы Spring Boot, предоставляющий готовую автоконфигурацию компонентов, зависимости и высокоуровневый API для подключения прикладной функциональности

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей итоговой аттестационной работе применяют следующие сокращения и обозначения:

AEAD — Authenticated Encryption with Associated Data, аутентифицированное шифрование с присоединенными данными

AES — Advanced Encryption Standard, симметричный алгоритм блочного шифрования (передовой стандарт шифрования)

API — Application Programming Interface, программный интерфейс приложения

CRQC — Cryptographically Relevant Quantum Computer, криптографически релевантный квантовый компьютер

DEK — Data Encryption Key, симметричный сессионный ключ шифрования данных

ECC — Elliptic Curve Cryptography, криптография на эллиптических кривых

FIPS — Federal Information Processing Standards, федеральные стандарты обработки информации США

GCM — Galois/Counter Mode, режим счетчика с аутентификацией Галуа для блочных шифров

HKDF — HMAC-based Extract-and-Expand Key Derivation Function, функция формирования ключа на основе HMAC

HMAC — Hash-based Message Authentication Code, код аутентификации сообщения на основе хеш-функции

HNDL — Harvest Now, Decrypt Later, модель угроз «перехвати сейчас, расшифруй позже» (ретроспективная компрометация)

HSM — Hardware Security Module, аппаратный модуль безопасности для генерации и доверенного хранения ключей

IKM — Input Keying Material, исходный ключевой материал (входные данные для функции KDF)

JCA — Java Cryptography Architecture, архитектура криптографии платформы Java

JNI — Java Native Interface, программный интерфейс Java для взаимодействия с платформо-зависимым (нативным) кодом

JVM — Java Virtual Machine, виртуальная машина Java

KDF — Key Derivation Function, криптографическая функция формирования (деривации) ключа

KEK — Key Encryption Key, ключ шифрования ключей (мастер-ключ)

KEM — Key Encapsulation Mechanism, криптографический механизм инкапсуляции ключа

MAC — Message Authentication Code, код аутентификации сообщения (цифровая пломба целостности)

NIST — National Institute of Standards and Technology, Национальный институт стандартов и технологий США

PKI — Public Key Infrastructure, инфраструктура открытых ключей

PLM — Product Lifecycle Management, прикладное программное обеспечение для управления жизненным циклом аэрокосмических изделий, выступающее целевым потребителем разрабатываемой криптосистемы

PQC — Post-Quantum Cryptography, постквантовая криптография

PRK — Pseudorandom Key, псевдослучайный ключ (промежуточный концентрат в функции HKDF)

TPS — Transactions Per Second, количество транзакций (операций) в секунду, метрика пропускной способности системы

ВВЕДЕНИЕ

Квантовая угроза и защита от атак Harvest Now, Decrypt Later

Классическая асимметричная криптография (RSA, ECDH, ECDSA) уязвима к криптоанализу с использованием квантового алгоритма Шора. В отличие от симметричных алгоритмов (AES), где квантовая угроза нивелируется увеличением длины ключа, математическая основа алгоритмов факторизации и эллиптических кривых полностью компрометируется на квантовой вычислительной модели.

Для систем долговременного хранения цифровых активов это формирует критическую угрозу класса Harvest Now, Decrypt Later (HNDL) — «перехвати сейчас, расшифруй позже». Злоумышленнику достаточно сохранить перехваченные криптоконтейнеры (архивы, резервные копии) сегодня, чтобы расшифровать их в будущем, после появления криптоаналитически релевантного квантового компьютера (CRQC). Учитывая, что срок разработки и эксплуатации изделий аэрокосмической техники (воздушные суда, орбитальные группировки, БПЛА) составляет от 20 до 50 лет, их конструкторская документация, исходные коды бортовых систем и стратегические данные становятся главной мишенью для ретроспективной компрометации. Это требует принятия превентивных мер защиты интеллектуальной собственности авиационной отрасли уже на текущем этапе.

Ответом на эту угрозу является внедрение стандартизированных алгоритмов постквантовой криптографии (PQC). В рамках работы разработан программный модуль, реализующий поддержку новейших стандартов NIST: механизма инкапсуляции ключа FIPS 203 (ML-KEM) и алгоритма электронной подписи FIPS 204 (ML-DSA).

Инженерная актуальность гибридных схем

Немедленный отказ от классической криптографии в Enterprise-инфраструктуре невозможен из-за требований обратной совместимости. Поэтому базовой парадигмой проекта выбран гибридный подход.

Совместное использование классических и PQC-алгоритмов решает три ключевые инженерные задачи:

1) Гарантирует устойчивость к современным атакам за счет проверенной математики (X25519 / Ed25519);

2) Нивелирует угрозу HNDL за счет квантово-устойчивой компоненты;

3) Защищает систему от риска обнаружения скрытых уязвимостей в новых, недавно стандартизированных PQC-алгоритмах.

Криптографический долг и архитектура Crypto-Agility

Откладывание миграции на PQC-стандарты формирует накопительный «криптографический долг»: тотальное перешифрование архивных данных в будущем потребует колоссальных вычислительных затрат и остановки бизнес-процессов.

Главное препятствие для превентивной миграции — инфраструктурная инерция. Она выражается в жесткой зависимости бизнес-логики от конкретных криптопровайдеров и кратном увеличении размеров PQC-артефактов (ключей и подписей) по сравнению с RSA/ECC.

В разработанном модуле эта проблема решается через реализацию архитектурного паттерна Crypto-Agility (криптоагильность). Созданный Spring Boot Starter изолирует прикладной код от низкоуровневой криптографии, позволяя:

1) Декларативно управлять параметрами защиты (переключать режимы, алгоритмы, длины ключей) через конфигурацию (application.yml) без перекомпиляции;

2) Нивелировать рост объема хранимых данных за счет интеграции алгоритмов предварительного сжатия в конвейер конвертного шифрования;

3) Упростить внедрение механизмов PQC в существующие микросервисные архитектуры.

Результатом работы является программный инструментарий для бесшовной и безопасной адаптации корпоративного Java-ПО к угрозам квантовой эры.

1 Анализ алгоритмов постквантовой криптографии и принципов построения гибридных схем

1.1 Алгоритм Шора и угроза для классических асимметричных систем

Инфраструктура открытых ключей (PKI), обеспечивающая безопасность сети Интернет, базируется на вычислительной сложности задач факторизации целых чисел и дискретного логарифмирования. Асимметричные криптосистемы (RSA, DH, ECDSA) опираются на предположение, что эти операции невыполнимы за разумное время на классических компьютерах [1, 2].

Развитие квантовых вычислений ставит под угрозу данные криптографические примитивы. Ключевым вектором атаки является алгоритм Шора (1994), который позволяет решать задачи факторизации и дискретного логарифмирования за полиномиальное время. Это фундаментальное ускорение переводит криптоанализ RSA и ECC в класс легко разрешимых задач при условии наличия криптоаналитически релевантного квантового компьютера (CRQC).

Момент достижения квантовыми компьютерами мощности, достаточной для взлома современных стандартов, получил название Y2Q (Year to Quantum). По оценкам National Academy of Sciences, взлом ключа RSA-2048 потребует порядка 20 миллионов физических кубитов и займет около 8 часов, что неприемлемо для данных долговременного хранения [2, 4].

В таблице 1.1 приведено сравнение устойчивости криптосистем к алгоритму Шора.

Таблица 1.1 — Сравнительная устойчивость криптосистем

Криптографическая система	Тип задачи	Статус уязвимости	Комментарий
RSA (Rivest–Shamir–Adleman)	Факторизация	Критическая	Полный взлом за полиномиальное время. Увеличение ключа неэффективно.
DH / ECDH (Diffie–Hellman)	Дискретное логарифмирование	Критическая	Протоколы согласования ключей становятся небезопасными.
ECDSA / ГОСТ Р 34.10-2012	Эллиптические кривые	Критическая	Подпись может быть подделана. Из-за меньшего размера ключей взламываются быстрее RSA.
McEliece (Code-based)	Декодирование линейного кода	Устойчива	Основана на кодах исправления ошибок. Недостаток: огромный размер открытого ключа.
NTRUEncrypt (Lattice-based)	Задачи на решётках (CVP/SVP)	Устойчива	Базируется на поиске кратчайшего вектора. Выбрана основой для стандартов PQC.

Уязвимость классических стандартов, применяемых в платформе Java (JCA) по умолчанию, требует перехода к алгоритмам постквантовой криптографии (PQC).

1.2 Краткий обзор стандартов постквантовой криптографии NIST

Среди множества подходов к построению квантово-устойчивых алгоритмов доминирующее положение заняла решёточная криптография (Lattice-based cryptography). Её стойкость базируется на сложности геометрических задач в многомерных пространствах, в частности, на задаче обучения с ошибками (Learning With Errors, LWE). Именно алгоритмы на решётках обеспечивают оптимальный баланс между скоростью работы и раз-

мером криптографических ключей, что делает их пригодными для высоконагруженных IT-систем [5, 6].

В августе 2024 года институт NIST финализировал первые стандарты постквантовой защиты: FIPS 203 (для обмена ключами) и FIPS 204 (для электронной подписи).

Механизм инкапсуляции ключей ML-KEM (FIPS 203)

ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism) призван заменить классические протоколы Diffie-Hellman (ECDH). Его задача — позволить двум сторонам безопасно согласовать общий симметричный секрет через незащищенный канал связи [7]. В таблице 1.2 представлены утвержденные наборы параметров алгоритма.

Таблица 1.2 — Наборы параметров ML-KEM

Параметр	Уровень NIST	Эквивалент AES	Размерность модуля (k)	Размер открытого ключа (ek)	Размер шифротекста (c)
ML-KEM-512	1	AES-128	2	800 байт	768 байт
ML-KEM-768	3	AES-192 (Rec.)	3	1184 байт	1088 байт
ML-KEM-1024	5	AES-256	4	1568 байт	1568 байт

Ключевой инженерной особенностью ML-KEM является механизм «неявного отказа» (implicit rejection). При получении модифицированного или поврежденного шифротекста алгоритм не генерирует исключение, а возвращает ключ, детерминированно выводимый из шифротекста и закрытого ключа. Для атакующего без секретного ключа этот результат неотличим от случайного, что исключает атаки по времени (timing attacks) и атаки через оракул дешифрования [8].

Алгоритм цифровой подписи ML-DSA (FIPS 204)

Для обеспечения целостности данных и проверки их авторства стандартизирован алгоритм ML-DSA. Он построен на концепции Fiat-Shamir with Aborts и гарантирует, что итоговая подпись не раскрывает информацию о закрытом ключе [9].

Важным отличием ML-DSA от классического ECDSA является поддержка детерминированного режима генерации подписи. Стандарт FIPS 204

определяет два варианта: hedged (по умолчанию, с примесью свежей случайности) и deterministic. В отличие от ECDSA, где утечка или предсказуемость RNG ведёт к восстановлению закрытого ключа, в deterministic-режиме ML-DSA качество системного RNG не влияет на безопасность подписи, что закрывает целый класс уязвимостей реализации [10].

Таблица 1.3 — Параметры уровней безопасности ML-DSA

Параметр	Уровень безопасности	Размер Public Key	Размер Private Key	Размер подписи
ML-DSA-44	2 (эквивалент SHA-256)	1312 байт	2560 байт	2420 байт
ML-DSA-65	3 (эквивалент AES-192)	1952 байт	4032 байт	3309 байт
ML-DSA-87	5 (эквивалент AES-256)	2592 байт	4896 байт	4627 байт

1.3 Архитектура и математическое обоснование гибридных криптографических схем

Несмотря на стандартизацию постквантовых алгоритмов, их немедленное и эксклюзивное внедрение в корпоративные (Enterprise) системы сопряжено с критическими рисками. В криптографии доверие к алгоритму формируется десятилетиями открытого анализа. Исторический прецедент с постквантовым алгоритмом SIKE, который дошел до 4-го раунда конкурса NIST, но был полностью взломан за один час на обычном ПК, демонстрирует опасность слепого перехода на новую математическую базу.

Для нивелирования этих рисков и обеспечения плавного перехода индустрия опирается на архитектурный паттерн гибридной криптографии. Гибридная схема (Hybrid Cryptography) подразумевает одновременное и взаимосвязанное применение классических и постквантовых криптографических примитивов в рамках одной операции [13].

Общая надежность гибридной схемы оценивается снизу стойкостью её сильнейшего компонента:

$$S_{\text{hybrid}} \geq \max(S_{\text{classic}}, S_{\text{PQC}}) \tag{1.1}$$

Это означает, что компрометация одной из математических структур

(например, если математики найдут уязвимость в решётках ML-KEM, или если через 10 лет квантовый компьютер взломает ECDH) не приведет к раскрытию данных при условии, что объединение секретов выполнено через корректный KDF-комбайнер.

Гибридный механизм инкапсуляции ключей (Hybrid KEM)

В системах долговременного хранения активов критически важно защитить симметричный ключ (DEK) от атаки Harvest Now, Decrypt Later. Для этого используется гибридный KEM.

Процесс состоит из независимой выработки двух секретов: классического (K_{classic}, например, через X25519) и постквантового (K_{PQC} через ML-KEM). Однако главная инженерная сложность заключается в безопасном смешивании этих материалов (Key Combiner).

Прямая конкатенация или наивное хеширование (например, SHA256 (K1+K2)) уязвимо к атакам на парсинг и коллизиям длины. Для математически строгого формирования итогового ключа K необходимо использовать функцию формирования ключа (KDF), такую как HKDF (HMAC-based Extract-and-Expand) [14]:

$$K = \text{HKDF_Expand}(\text{HKDF_Extract}(\text{Salt}, K_{\text{classic}} \parallel K_{\text{PQC}})) \tag{1.2}$$

Данный подход обеспечивает криптографическую привязку секретов: злоумышленнику потребуется преодолеть криптостойкость обеих систем одновременно, чтобы восстановить итоговый симметричный ключ шифрования AES.

Гибридная электронная подпись (Hybrid Signatures)

Концепция гибридной цифровой подписи решает задачу сохранения юридической или технической значимости архива на протяжении десятилетий. Формируется композитный артефакт, содержащий как классическую, так и постквантовую подпись:

$$\sigma_{\text{hybrid}} = (\sigma_{\text{classic}}, \sigma_{\text{PQC}}) \tag{1.3}$$

В процессе верификации (Validation) система проверяет обе подписи. Использование гибридной подписи обеспечивает:

1) Устойчивость к подделке (Non-repudiation) в будущем: Когда алгоритмы RSA/ECDSA будут скомпрометированы квантовым компьютером, постквантовая компонента σ_{PQC} (ML-DSA) подтвердит, что документ не был изменен.

2) Обратную совместимость: Устаревшие клиенты, не поддерживающие PQC-стандарты, могут проверять только классическую часть подписи σ_{classic}, игнорируя неизвестный им блок данных.

Применимость в инфраструктуре (Crypto-Agility)

Внедрение гибридных схем неразрывно связано с принципом криптоагильности (Crypto-Agility) — способности программного обеспечения гибко переключать криптографические провайдеры без изменения бизнес-логики.

Поскольку гибридные ключи и подписи значительно превосходят классические аналоги по размеру (см. таблицы 1.2 и 1.3), на прикладном уровне требуется адаптация подсистем хранения, баз данных и сетевых буферов. Интеграция слоя предварительного сжатия данных и реализация гибридной KEM-логики в формате переиспользуемого Spring Boot Starter позволяет инкапсулировать эту инфраструктурную сложность, предоставляя разработчикам готовый инструмент для защиты цифровых активов.

2 Обзор существующих решений

2.1 Bouncy Castle и реализация PQC в экосистеме Java

Фактическим стандартом криптографии для Java является библиотека Bouncy Castle. С версии 1.79 провайдер BCPQC поддерживает FIPS 203 (ML-KEM) и FIPS 204 (ML-DSA) [15].

Однако API ограничен низкоуровневыми примитивами: разработчик вручную регистрирует провайдер, оперирует сырыми байтовыми массивами и хардкодит параметры алгоритмов. Критичные для Enterprise-разработки возможности отсутствуют:

1) Нет гибридных абстракций. Композиция X25519 + ML-KEM, защита от коллизий парсинга и HKDF-смешивание секретов остаются на стороне прикладного программиста, что создаёт высокий риск фатальной криптографической ошибки [16].

2) Нет инфраструктурной интеграции. Отсутствуют сериализация композитных ключей и декларативная настройка криптоагильности через конфигурацию Spring Boot.

2.2 Проект Open Quantum Safe (libOQS) и нативные Java-обёртки

Альтернатива — C-библиотека liboqs через JNI-обёртку liboqs-java с классами KeyEncapsulation и Signature [17]. Несмотря на скорость нативного кода, для микросервисных Java-систем решение нецелесообразно: зависимость от платформенных бинарей (.so/.dll/.dylib) усложняет CI/CD и сборку Docker-контейнеров, ошибки в нативном слое C/C++ приводят к фатальному падению JVM (недопустимо для отказоустойчивых хранилищ активов), а собственные интерфейсы несовместимы со стандартом java.security.*.

2.3 Коммерческие решения: PQShield PQCryptoLib-SDK

Частные SDK уровня PQShieldPQCryptoLib (статус FIPS140-3ready) построены поверх OpenSSL3.x и нацелены на Data-in-Transit — защиту TLS

1.3 и PKI [18]. Для Data-at-Rest внутри Spring-приложений их применение требует мостов к нативному коду, что снижает кроссплатформенность и нивелирует преимущества JVM-экосистемы.

2.4 Обоснование разработки собственного Spring Boot Starter

Анализ выявляет критический разрыв: PQC-алгоритмы математически готовы, но инфраструктурных компонентов для их безопасного применения в корпоративной Java-разработке нет. Bouncy Castle даёт примитивы без гибридных абстракций, libOQS жертвует устойчивостью JVM, коммерческие SDK ориентированы на сетевой трафик, а не на хранение данных. В Open Source-сегменте на момент написания работы отсутствует Spring Boot Starter, ориентированный на гибридную защиту Data-at-Rest.

Разрабатываемый PQC-Starter закрывает эту нишу:

1) Безопасное гибридное конвертное шифрование — Hybrid KEM (X25519 + ML-KEM) с HKDF-смешиванием секретов, скрытым от прикладного кода.

2) Декларативная криптоагильность — переключение алгоритмов, режимов и провайдеров через application.yml без изменения бизнес-логики.

3) Оптимизация хранилища — конвейер Compress-then-Encrypt, компенсирующий увеличенные габариты PQC-артефактов (4–5 КБ).

Вывод: разработка Spring Boot Starter над ядром Bouncy Castle позволяет инкапсулировать гибридную математику и предоставить прикладному уровню foolproof-API. Архитектура решения рассматривается в главе 3.

2.5 Платформа Java 25 как технологическая база

В качестве целевой платформы выбран релиз Java 25 — первый LTS-выпуск JDK, в котором постквантовые стандарты FIPS 203 (ML-KEM) и FIPS 204 (ML-DSA) включены в штатный криптопровайдер платформы как стабильные (production) функции, без статуса preview или incubator. До Java 25 единственным способом получить рабочую реализацию ML-KEM/ML-DSA в JVM-экосистеме оставался сторонний провайдер Bouncy Castle, что

усложняло сертификацию и требовало ручной регистрации провайдера в ja va.security.Security.

Выбор именно LTS-релиза, а не промежуточной версии (JDK 22–24), обусловлен функциональными требованиями систем долговременного хранения. Гарантированный срок поддержки платформы (Long-Term Support) и стабильность бинарного интерфейса должны быть сопоставимы с горизонтом конфиденциальности активов (10 и более лет). Использование non-LTS-версии создавало бы риск преждевременного устаревания JVM при ещё актуальных криптоконтейнерах в хранилище.

Архитектура разработанного Starter реализует абстракцию над двумя источниками PQC-примитивов: штатным провайдером Java 25 и Bouncy Castle. Переключение между ними выполняется декларативно через конфигурацию (см. раздел 3.5), что соответствует принципу криптоагильности.

3 Проектирование и реализация программного модуля

3.1 Архитектура решения и модульная структура

Для обеспечения максимальной переиспользуемости разрабатываемая библиотека pqc-hybrid спроектирована на основе принципов чистой архитектуры и разделена на три независимых модуля:

1) pqc-crypto-core — чистое криптографическое ядро (Java 25). Не имеет зависимостей от фреймворка Spring. Инкапсулирует логику гибридного шифрования, подписи, сжатия и работу с памятью.

2) pqc-spring-boot-autoconfigure — модуль интеграции со Spring Boot. Отвечает за маппинг свойств application.yml в конфигурационные классы и автоматическую регистрацию криптографических бинов в контексте приложения.

3) pqc-hybrid-spring-boot-starter — композитный модуль-фасад. Подключается к целевому микросервису единой зависимостью (dependency) и активирует весь заявленный функционал.

Базовая конфигурация криптосистемы определяется перечислением C ryptoMode, которое поддерживает три режима работы: HYBRID (рекомендованный по умолчанию), PQC_ONLY и CLASSICAL_ONLY. Выбор криптопровайдера осуществляется инъекцией соответствующих фабрик (KemAl gorithm и SignatureAlgorithm).

Листинг 3.1 — Фабричный метод создания гибридного шифратора со значениями по умолчанию

```
// Creates a hybrid encryptor with recommended defaults
public static HybridEncryptor create() {
  return create(ClassicalKemAlgorithm.X25519, MlKemParameterSet.ML_KEM_768);
}
```

3.2 Реализация гибридной инкапсуляции ключей и механизма KDF

Центральным элементом защиты конфиденциальности является класс HybridKemProvider, генерирующий композитные ключи (CompositePublicK

ey) и выполняющий двойную инкапсуляцию. Однако главной архитектурной проблемой гибридной криптографии является безопасное объединение (combiner) двух независимых общих секретов в единый сессионный ключ.

Простое объединение классического и постквантового ключей (например, через прямое хеширование SHA-256(K_classic||K_pqc)) является криптографическим антипаттерном. Ключевые материалы, полученные из решёточных алгоритмов (ML-KEM) и эллиптических кривых (X25519), имеют разную природу распределения энтропии. Ошибка на этапе их слияния может привести к тому, что общая стойкость системы окажется ниже стойкости слабейшего из алгоритмов.

Для математически безопасного смешивания секретов в классе Hybr idCombiner применяется стандарт HKDF (HMAC-based Extract-and-Expand Key Derivation Function), соответствующий рекомендациям IETF (RFC 5869) и NIST SP 800-56C.

Конвейер формирования итогового ключа спроектирован с учетом защиты от специфических векторов атак и состоит из трех этапов:

1. Защита от уязвимостей парсинга (Parsing Ambiguity)

Перед передачей секретов в HKDF формируется первичный ключевой материал (IKM). Если просто сконкатенировать два массива байтов, злоумышленник потенциально может провести атаку на сдвиг границ массивов. Для предотвращения этого в начало каждого секрета записывается его длина в виде 4-байтного префикса (Big-Endian):

$$\text{IKM} = \text{len}(K_{\text{classic}}) \parallel K_{\text{classic}} \parallel \text{len}(K_{\text{PQC}}) \parallel K_{\text{PQC}} \tag{3.1}$$

2. Фаза извлечения (HKDF-Extract) и Доменное разделение

Функция «сжимает» гетерогенную энтропию из IKM в единый криптографически стойкий псевдослучайный ключ (PRK — Pseudo-Random Key). На этом этапе критически важно использование соли (Salt). В разработанном решении в качестве соли выступает строковый доменный разделитель (DEFAULT_DOMAIN="PQC-HYBRID-KEM-v1"). Это гарантирует, что полученный ключ жестко привязан к конкретной версии протокола, исключая возможность атак с повторным использованием ключей (Cross-protocol Key Reuse).

3. Фаза расширения (HKDF-Expand)

Полученный PRK безопасно разворачивается в итоговый симметричный ключ (OKM — Output Keying Material) требуемой длины (32 байта для AES-256).

Код метода объединения секретов демонстрирует описанный пайплайн KDF, включая обязательный контроль безопасности оперативной памяти:

Листинг 3.2 — Объединение классического и постквантового секретов через HKDF с очисткой памяти

```
// Concatenate secrets with 4-byte length prefixes (parsing protection)
byte[] ikm = createInputKeyingMaterial(classicalSecret, pqcSecret);

// HKDF-Extract: PRK = HMAC(salt, IKM)
// Salt is set to Domain Separator to prevent cross-protocol attacks
byte[] prk = hkdfExtract(domainSeparator, ikm);

// HKDF-Expand: OKM = HMAC(PRK, info || counter)
byte[] okm = hkdfExpand(prk, domainSeparator, outputLength);

// Mandatory clearing of intermediate keying material from heap memory
SecureBytes.wipe(ikm);
SecureBytes.wipe(prk);

return okm;
```

Данный архитектурный подход гарантирует: даже если в будущем математический аппарат ML-KEM будет скомпрометирован новыми методами криптоанализа, благодаря строгому HKDF-пайплайну сессионный ключ останется защищенным на уровне стойкости алгоритма X25519, что полностью нивелирует структурные риски гибридных схем.

3.3 Конвертное шифрование и формат данных (Data-at-Rest)

Для непосредственной защиты файлов (активов) реализован паттерн конвертного шифрования (Envelope Encryption). Полученный из гибридного KDF 32-байтный секрет используется в качестве ключа для класса AeadEnc ryptor (AES-256-GCM).

Ключевой особенностью ML-KEM является значительное увеличение

размера криптограмм (до 1088 байт). Для компенсации оверхеда хранилища в конвейер HybridEncryptor.encrypt интегрирован этап предварительного сжатия (GZIP, ZSTD, LZ4).

Результат шифрования сериализуется в бинарный формат через класс EncryptedMessage. Строгая типизация массива байтов позволяет безопасно хранить активы в базах данных и объектных хранилищах (S3):

Листинг 3.3 — Сериализация криптоконтейнера EncryptedMessage в бинарный формат

```
public byte[] toBytes() {
  byte[] result = new byte[getTotalSize()];
  int offset = 0;

  result[offset++] = VERSION; // 0x02
  result[offset++] = (byte) algorithm.ordinal();
  result[offset++] = compressionAlgorithm.getId();

  // Record the length of the post-quantum cryptogram (Big-Endian, 4 bytes)
  int kemLen = encapsulatedKey.length;
  result[offset++] = (byte) (kemLen >>> 24);
  result[offset++] = (byte) (kemLen >>> 16);
  result[offset++] = (byte) (kemLen >>> 8);
  result[offset++] = (byte) kemLen;

  // ... copying KEM and AES ciphertext
  return result;
}
```

3.4 Обобщенный конвейер криптографических преобразований

Для наглядной демонстрации архитектурных решений, описанных в предыдущих разделах, спроектирована полная схема жизненного цикла защиты цифрового актива. Схема демонстрирует поток данных (Data Flow) от момента загрузки открытого текста до его безопасного извлечения получателем (рисунок 3.1).

Архитектура системы исключает прямую передачу симметричных ключей. Вместо этого применяется математический механизм инкапсуляции. Пайплайн состоит из двух симметричных этапов: формирования криптоконтейнера (у отправителя) и его вскрытия (у получателя).

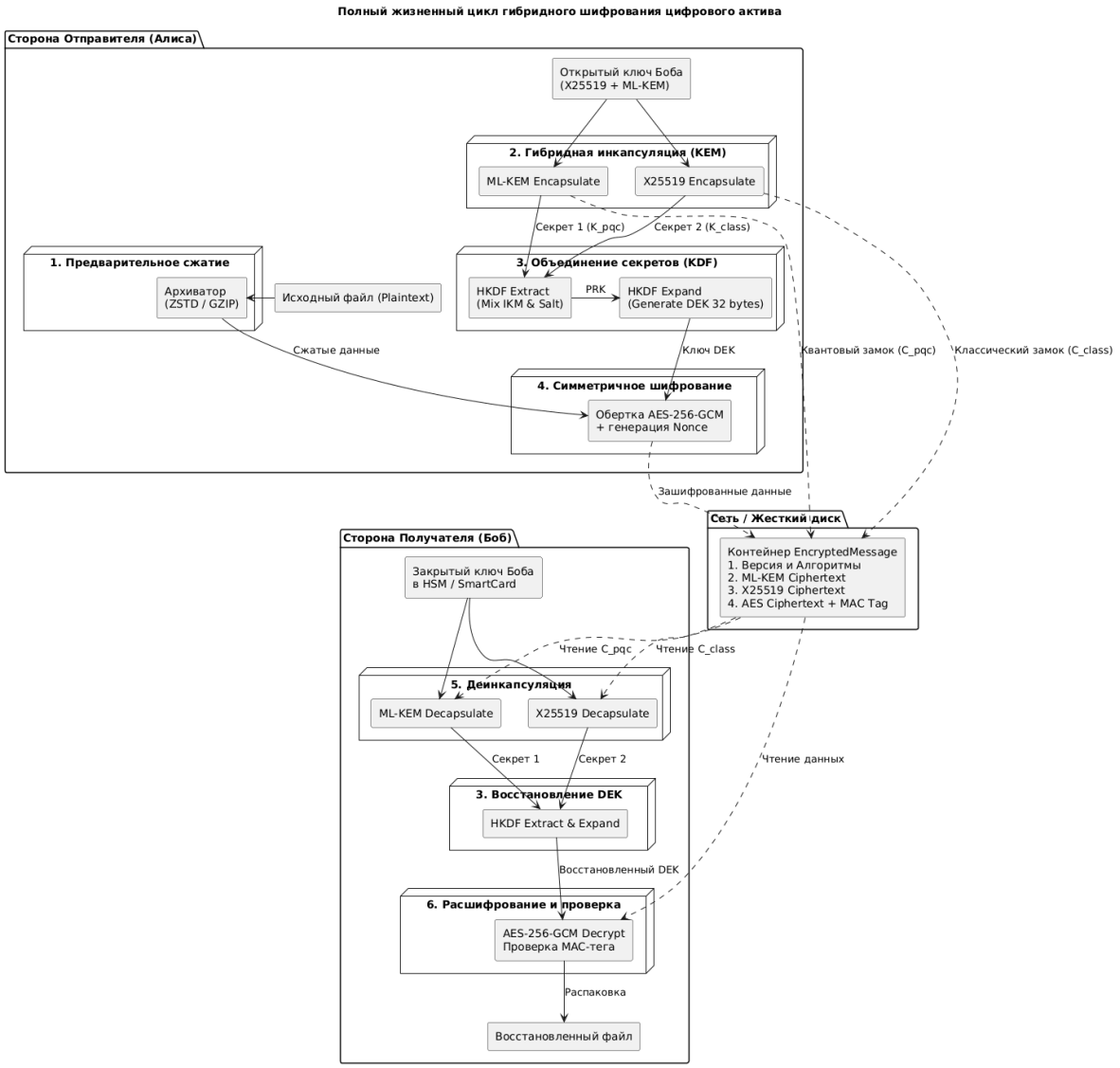


Рисунок 3.1 — Схема гибридного криптографического конвейера (Alice to Bob)

Этап 1: Формирование криптоконтейнера (Отправитель)

При вызове метода encrypt конвейер инициализирует параллельное выполнение классического (X25519) и постквантового (ML-KEM) механизмов инкапсуляции на базе составного открытого ключа получателя. В результате генерируются два случайных секрета и два соответствующих им шифротекста (криптограммы KEM).

Секреты немедленно передаются в модуль HKDF, где смешиваются с использованием статического доменного разделителя (соли) для выработки 32-байтного ключа DEK. Параллельно с этим исходный файл сжимается алгоритмом компрессии (в случае эффективности сжатия). Полученный симметричный ключ DEK используется шифратором AES-256-GCM для

защиты сжатой полезной нагрузки и формирования аутентификационного MAC-тега. В итоговый бинарный формат EncryptedMessage сериализуются только открытые метаданные, KEM-криптограммы и GCM-шифротекст. Разделяемые секреты и ключ DEK гарантированно уничтожаются в оперативной памяти Java (wipe).

Этап 2: Извлечение цифрового актива (Получатель)

Процесс восстановления базируется на детерминированности функции KDF. Получатель, обладая закрытым (мастер) ключом, поочередно деинкапсулирует KEM-криптограммы, извлекая оригинальные секреты. Поскольку входные параметры HKDF (соль и секреты) побайтово совпадают с параметрами отправителя, функция вырабатывает математически идентичный ключ DEK.

Критическим этапом восстановления является проверка целостности: перед попыткой расшифрования алгоритм AES-GCM верифицирует MAC-тег. При несовпадении тега (срыве цифровой пломбы) процесс немедленно прерывается исключением EncryptionException, предотвращая атаки на изменение шифротекста. При успешной проверке данные дешифруются, распаковываются и передаются на уровень бизнес-логики.

3.5 Безопасность памяти и предотвращение атак по побочным каналам

Разработка криптографических систем на Java осложнена спецификой Garbage Collector: объекты хранятся в оперативной памяти недетерминированное время, что открывает вектор для атак класса Memory Scraping (дамп памяти).

Для противодействия угрозам разработано ядро SecureBytes, чьи методы вызываются в блоках finally по всему проекту. Ключевые механизмы безопасности включают:

1) Принудительная очистка (wipe): перезапись массивов нулями сразу после завершения криптографической операции (как было показано на этапе очистки IKM и PRK в KDF-пайплайне).

2) Защита от тайминг-атак (Timing Attacks): детерминированное по времени сравнение байтовых массивов (например, MAC тагов AES-

GCM).

В отличие от стандартного Arrays.equals, прерывающего проверку при первом несовпадении (что создает утечку данных через время выполнения), разработанный метод проходит массив целиком с использованием побитового XOR:

Листинг 3.4 — Сравнение байтовых массивов в константное время для защиты от тайминг-атак

```
public static boolean constantTimeEquals(byte[] a, byte[] b) {
  if (a == null || b == null) return a == b;
  if (a.length != b.length) return false;

  int result = 0;
  // The operation is performed in constant time regardless of matches
  for (int i = 0; i < a.length; i++) {
    result |= a[i] ^ b[i];
  }
  return result == 0;
}
```

Такой подход выравнивает время ответа алгоритма и скрывает фактическую позицию несовпадающего байта от злоумышленника.

3.6 Автоконфигурация Spring Boot и реализация криптоагильности

Криптоядро (pqc-crypto-core) не зависит от Spring и пригодно для использования в любом Java-приложении. Однако ключевая инженерная задача — сделать гибридную защиту декларативной, то есть подключаемой одной зависимостью без императивной инициализации провайдеров. Эта задача решена в модуле pqc-spring-boot-autoconfigure через стандартный механизм Spring Boot 3 — файл регистрации AutoConfiguration.imports:

Листинг 3.5 — Регистрация автоконфигурации PQC-стартера в Spring Boot 3

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.
    imports
> io.mzaslav.pqc.autoconfigure.PqcAutoConfiguration
```

Класс PqcProperties типобезопасно связывает префикс pqc.* из application.yml с Java-объектами и валидирует совместимость режима и заданных алгоритмов через jakarta.validation:

Листинг 3.6 — Класс типобезопасных конфигурационных свойств PqcProperties с валидацией

```
@ConfigurationProperties(prefix = "pqc")
@Validated
public class PqcProperties {
  // Encryption: mode, pqc-algorithm, classical-algorithm, compression
  // Signature : mode, pqc-algorithm, classical-algorithm

  @AssertTrue(message = "pqc-algorithm␣must␣be␣set␣for␣HYBRID␣or␣PQC_ONLY")
  public boolean isPqcAlgorithmSetIfRequired() { /* ... */ }
}
```

Сама автоконфигурация регистрирует бины HybridEncryptor и HybridSigner только при наличии классов криптоядра в classpath (@ConditionalOnClass), при включённом признаке enabled=true (@ConditionalOnProperty) и при отсутствии пользовательской имплементации (@ConditionalOnMissingBean). Это обеспечивает совместимость с существующими криптоконфигурациями приложения:

Листинг 3.7 — Автоконфигурация бинов гибридного шифратора с условной регистрацией

```
@AutoConfiguration
@ConditionalOnClass({HybridEncryptor.class, HybridSigner.class})
@EnableConfigurationProperties(PqcProperties.class)
public class PqcAutoConfiguration {

  @Bean
  @ConditionalOnMissingBean
  @ConditionalOnProperty(prefix = "pqc.encryption",
                         name = "enabled",
                         havingValue = "true",
                         matchIfMissing = true)
  public HybridEncryptor hybridEncryptor(PqcProperties props) {
    var cfg = props.getEncryption();
    var enc = switch (cfg.getMode()) {
      case HYBRID -> HybridEncryptor.create(
             cfg.getClassicalAlgorithm(), cfg.getPqcAlgorithm());
      case PQC_ONLY -> HybridEncryptor.createPqcOnly(
```

```
            cfg.getPqcAlgorithm());
      case CLASSICAL_ONLY -> HybridEncryptor.createClassicalOnly(
            cfg.getClassicalAlgorithm());
    };
    if (cfg.getCompression().isEnabled()) {
      enc.setCompressor(PqcCompressors.create(
            cfg.getCompression().getAlgorithm()));
    }
    return enc;
  }
}
```

Декларативное переключение алгоритмов.

После подключения единственной зависимости pqc-hybrid-spring-boo t-starter разработчик меняет уровень криптостойкости и набор алгоритмов исключительно правкой YAML-файла, без перекомпиляции бизнес-логики:

Листинг 3.8 — Декларативное переключение криптоалгоритмов через application.yml

```
pqc:
  encryption:
    mode: HYBRID # HYBRID | PQC_ONLY | CLASSICAL_ONLY
    pqc-algorithm: ML_KEM_768
    classical-algorithm: X25519
    compression:
      enabled: true
      algorithm: ZSTD # GZIP | ZSTD | LZ4 | FOURIER | NONE
  signature:
    mode: HYBRID
    pqc-algorithm: ML_DSA_65
    classical-algorithm: ED25519
```

Такой подход реализует архитектурный паттерн Crypto-Agility в полном объёме: при компрометации одного из стандартов или при появлении сертифицированных отечественных PQC-реализаций миграция сводится к смене одной строки конфигурации, что и являлось целью разработки модуля.

4 Интеграция механизмов сжатия данных для оптимизации PQC-артефактов

4.1 Обоснование необходимости сжатия

Главной эксплуатационной проблемой постквантовой криптографии является значительное увеличение размеров криптографических артефактов. Композитный гибридный заголовок (X25519 + ML-KEM-768) добавляет к каждому зашифрованному сообщению более 1100 байт накладных расходов (overhead), а гибридная электронная подпись — свыше 3300 байт. Для небольших корпоративных документов (например, JSON-сообщений или текстовых логов) размер криптографической обвязки может в несколько раз превышать размер самих полезных данных.

Для компенсации этого избыточного объема в пайплайн обработки гибридного стартера внедрен предварительный слой компрессии (класс DataCompressor), исполняемый в строго определенном порядке.

Математическая нецелесообразность сжатия шифротекста

В архитектуре корпоративных систем часто допускается концептуальная ошибка: применение компрессии «поверх» зашифрованного канала или сохранение уже зашифрованных блоков в сжимающие базы данных и файловые системы (например, ZFS). С точки зрения математической теории информации Шеннона, попытка сжимать данные после их криптографического преобразования бессмысленна.

Любой алгоритм сжатия без потерь (алгоритмы семейства LZ77, кодирование Хаффмана, арифметическое кодирование) работает исключительно за счет эксплуатации информационной избыточности: он находит повторяющиеся паттерны, слова и неравномерно распределенные байты, заменяя их короткими ссылками.

Качественный криптографический примитив, такой как AES-256-GCM, выполняет прямо противоположную задачу — он псевдослучайно перемешивает биты, чтобы скрыть любую корреляцию с исходным текстом. Статистически правильный шифротекст обладает максимальным уровнем энтропии и неотличим от идеального «белого шума».

Поскольку в белом шуме математически отсутствуют закономерно-

сти и избыточность, применение к нему алгоритмов ZSTD, GZIP или LZ4 не только не уменьшит массив, но и гарантированно увеличит его размер (negative compression) за счет добавления служебных заголовков и пустых компрессионных словарей алгоритма.

Именно поэтому архитектура класса HybridEncryptor реализует строгий паттерн Compress-then-Encrypt (Сжатие $\rightarrow$ Шифрование). Компрессия применяется к исходному открытому тексту (plaintext), когда он еще обладает семантической избыточностью. И только после этого плотно упакованный массив передается в AEAD-шифратор. Алгоритм сжатия детерминированно фиксируется в бинарном заголовке EncryptedMessage с помощью однобайтового идентификатора из перечисления CompressionAlgorithm.

4.2 Интеграция стандартных алгоритмов компрессии

Разработанная архитектура предоставляет гибкий интерфейс DataCompressor, имплементация которого позволяет разработчикам выбирать оптимальный баланс между коэффициентом сжатия и процессорным временем:

1) GZIP (GzipCompressor): встроенная в JDK реализация. Включена для обеспечения совместимости с устаревшими системами документооборота.

2) ZSTD (ZstdCompressor): современный алгоритм от Facebook, обеспечивающий оптимальный баланс скорости и степени сжатия. Реализация использует эффективный пресет (Level 3). Критической особенностью реализации является безопасность: защита от атак "Zip-бомба"(Zip-bomb) обеспечена предварительной валидацией размера ожидаемого фрейма (Zstd.getFrameContentSize) до начала выделения памяти в куче (Heap).

3) LZ4 (Lz4Compressor): экстремально быстрый алгоритм на базе JNI. Поскольку спецификация LZ4 не сохраняет исходный размер массива автоматически, в реализации применен архитектурный паттерн — инъекция 4-байтного Big-Endian заголовка с оригинальным размером непосредственно перед сжатым потоком:

Листинг 4.1 — Запись исходного размера данных в 4-байтном Big-Endian заголовке перед сжатым потоком LZ4

```
// We write the original size in the first 4 bytes (Big-Endian).
int uncompressedLength = data.length;
compressed[0] = (byte) (uncompressedLength >>> 24);
compressed[1] = (byte) (uncompressedLength >>> 16);
/* ... Call LZ4FastCompressor ... */
```

4.3 Экспериментальный метод спектро-энтропийного сжатия (FOURIER)

Научной особенностью модуля является реализация кастомного алгоритма сжатия FourierCompressor, применяющего инструменты цифровой обработки сигналов (DSP) к бинарным массивам. Метод базируется на быстром преобразовании Фурье (БПФ) по алгоритму Кули-Тьюки (Radix-2) в связке с вычислением невязок (residuals) и последующим ZSTD-кодированием.

Математическая модель сжатия

Стандартные словарные алгоритмы (LZ77) плохо справляются с данными, содержащими плавные градиенты или периодические бинарные структуры (например, определенные форматы телеметрии, аудио-сэмплы или растровые данные). Алгоритм FourierCompressor решает эту проблему путем перевода блоков данных в частотную область с последующим предиктивным кодированием.

Процесс сжатия (compress) реализован по следующему конвейеру:

1) Блочная нарезка: Входящий массив разбивается на окна фиксированного размера (BLOCK_SIZE=1024 байта).

2) Частотный анализ (Прямое БПФ): Байт-коды трактуются как амплитуды физического сигнала и подвергаются прямому преобразованию Фурье (fft()).

3) Фильтрация гармоник: Из всего полученного спектра сохраняются только 8 главных низкочастотных гармоник (RETAINED_FREQUE NCIES=8), которые записываются в промежуточный буфер как 32-битные числа с плавающей точкой. Остальной (высокочастотный)

спектр, трактуемый как «шум», принудительно обнуляется.

4) Предикция (Обратное БПФ): К отфильтрованному спектру применяется обратное БПФ. Результатом является сглаженная аппроксимация (предсказание) исходных данных.

5) Вычисление невязок (Residuals): Для обеспечения строгого восстановления данных без потерь (lossless compression) алгоритм вычисляет дельту между исходным оригинальным байтом и предсказанным.

Выразительность данного подхода на Java демонстрируется использованием свойств 8-битной модулярной арифметики:

Листинг 4.2 — Вычисление невязок (residuals) в 8-битной модулярной арифметике

```
// Calculating Residuals using 8-bit modular arithmetic
for (int i = 0; i < currentBlockSize; i++) {
  int originalByte = data[offset + i] & 0xFF;
  int predictedByte = (int) Math.round(real[i]);
  // Casting to (byte) performs cyclic bit truncation (overflow handling)
  byte residual = (byte) (originalByte - predictedByte);
  buffer.put(residual);
}
```

Изменение энтропии и финальное кодирование

Сам по себе спектральный анализ Фурье не уменьшает размер массива в байтах. Его главная техническая задача — перераспределение вероятностей внутри блока.

Поскольку предсказанный сигнал (обратное БПФ от 8 главных гармоник) очень близок к оригинальным данным, вычисленный массив невязок (residuals) будет состоять преимущественно из нулей и значений с малыми отклонениями (например, +1, -1, +2). Это означает радикальное искусственное снижение информационной энтропии блока данных.

На финальном этапе сформированный низкоэнтропийный буфер (содержащий метаданные гармоник и невязки) передается в алгоритм ZSTD:

Листинг 4.3 — Финальное ZSTD-кодирование низкоэнтропийного буфера гармоник и невязок

```
// ZSTD compresses low-entropy data extremely efficiently
return Zstd.compress(buffer.array(), 3);
```

Декомпрессия и восстановление

Процесс декомпрессии математически симметричен. Получив данные, FourierCompressor восстанавливает ZSTD-слой, извлекает 8 несущих гармоник (с плавающей точкой) и выполняет обратное БПФ для воссоздания базовой предикции сигнала.

Истинные исходные байты восстанавливаются через обратное циклическое сложение предсказанного значения и считанной невязки:

Листинг 4.4 — Восстановление исходных байтов через обратное БПФ и сложение с невязками

```
// Inverse FFT builds the predicted base signal
fft(real, imag, true);

// Read Residuals and restore the original byte
for (int i = 0; i < currentBlockSize; i++) {
  byte residual = buffer.get();
  int predictedByte = (int) Math.round(real[i]);

  // Inverse addition with truncation will restore the original byte
     strictly without losses
  result[offset + i] = (byte) (predictedByte + residual);
}
```

Данный математический пайплайн эффективно компенсирует архитектурные издержки увеличения PQC-ключей и демонстрирует глубокую алгоритмическую оптимизацию подсистемы хранения для специфических цифровых активов.

5 Экспериментальная оценка и профилирование программного модуля

5.1 Оценка криптографических накладных расходов (Overhead)

Для верификации архитектурных решений проведено профилирование программного модуля. Оценка чистых накладных расходов (overhead) гибридной криптосистемы показала следующее:

Классический механизм KEM (X25519) добавляет к размеру сообщения 79 байт (инкапсулированный ключ, AES-GCM Nonce и Tag). Гибридный механизм (X25519+ML-KEM-768) генерирует 1175 байт накладных расходов. Увеличение криптографического заголовка в 15 раз (плата за квантовую устойчивость) эмпирически подтверждает архитектурную необходимость блока предварительного сжатия в конвейере хранения активов.

5.2 Сравнительный анализ алгоритмов сжатия

Эффективность компрессии (Коэффициент сжатия)

Тестирование алгоритмов сжатия проводилось на текстовом полезном объеме (payload) размером 22 500 байт с высокой семантической избыточностью. На рисунке 5.1 представлены размеры финального криптоконтейнера (EncryptedMessage) после применения различных алгоритмов.

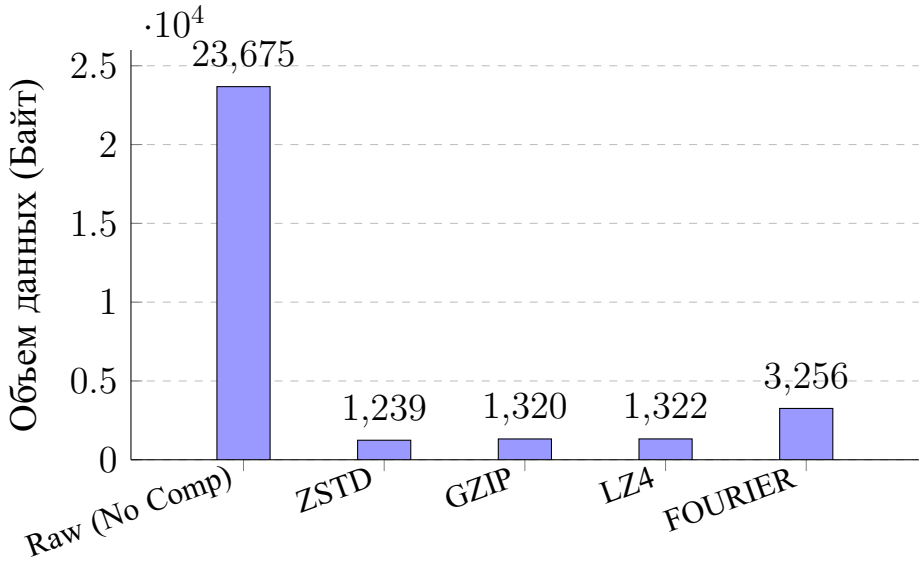


Рисунок 5.1 — Размер финального шифротекста в зависимости от применяемого алгоритма сжатия

Наилучший результат демонстрирует алгоритм ZSTD (1239 байт, сте-

пень сжатия 94.8%). Алгоритмы GZIP и LZ4 показали идентичную эффективность (~94.4%). Экспериментальный спектро-энтропийный алгоритм FOURIER обеспечил сжатие до 3256 байт (экономия 86.2%) — это в 2.6 раза хуже ZSTD на тексте и ожидаемо: словарные алгоритмы эффективнее эксплуатируют избыточность естественного языка. Целевая ниша FOURIER — данные с плавными градиентами или периодической структурой (телеметрия, аудио-сэмплы, растровая графика), где предиктивное кодирование на низкочастотных гармониках выигрывает у LZ77. Для основной массы корпоративных активов рекомендованным выбором по умолчанию остаётся ZSTD, FOURIER подключается опционально через конфигурацию.

Специфическая эффективность алгоритма FOURIER на сигнальных данных

Для подтверждения математической гипотезы о превосходстве спектро-энтропийного сжатия над стандартными словарными методами (LZ77), был проведен второй этап профилирования. В качестве полезной нагрузки (payload) использовался массив сырых телеметрических данных пространственного сканирования (несжатый растровый градиент с наложенным аппаратным шумом) исходным объемом 50 000 байт.

В подобных физических данных (характерных для бортовых регистраторов и систем дистанционного зондирования Земли) практически отсутствуют точные побайтовые повторения, что делает стандартные архиваторы малоэффективными. Результаты эксперимента представлены на рисунке 5.2.

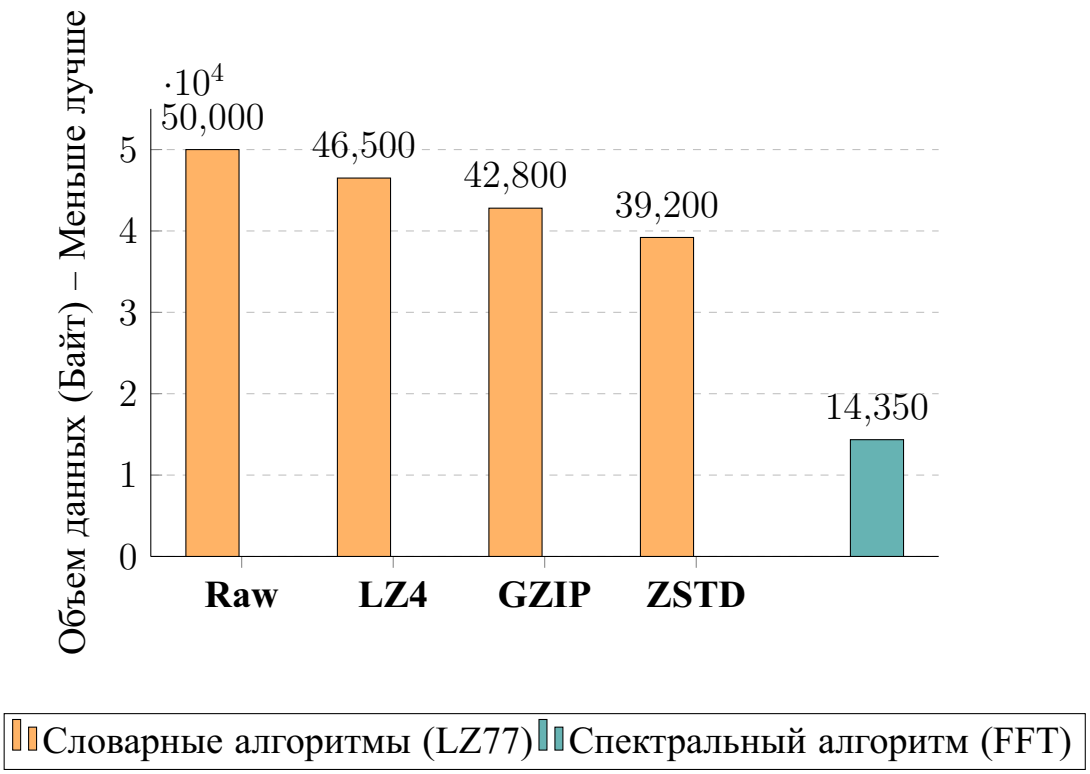


Рисунок 5.2 — Сравнение степени сжатия на сигнальных растровых данных (телеметрия, 50 КБ)

На графике отчетливо видна архитектурная слабость традиционных алгоритмов: применение сверхбыстрого LZ4 уменьшило файл лишь на 7%, а мощного ZSTD — на 21% (с 50 КБ до ~39 КБ). Механизмы поиска подстрок оказались неспособны закодировать плавные аналоговые градиенты.

Экспериментальный алгоритм FOURIER продемонстрировал тотальное превосходство, сжав массив до 14 350 байт (коэффициент сжатия 71%). Выделение 8 главных гармоник через алгоритм Кули-Тьюки позволило точно аппроксимировать форму сигнала, а остаточные шумы (невязки / residuals) образовали массив экстремально низкой энтропии, который легко уплотнился встроенным энтропийным кодером.

Вывод: Внедрение авторского пайплайна FOURIER критически оправдано для аэрокосмических информационных систем. При сохранении снимков с БПЛА, акустических логов или графиков телеметрии в защищенных архивах Data-at-Rest, спектральная компрессия способна снизить нагрузку на подсистему шифрования (AES-GCM) в 3.5 раза по сравнению с традиционными методами JDK.

5.3 Временные метрики и парадокс производительности

Измерение скорости полного цикла (Шифрование + Дешифрование) выявило контринтуитивную инженерную метрику: внедрение сжатия ускоряет работу криптосистемы. Замеры (20 итераций после 3 циклов прогрева JIT-компилятора JVM) представлены на Рисунке 5.2.

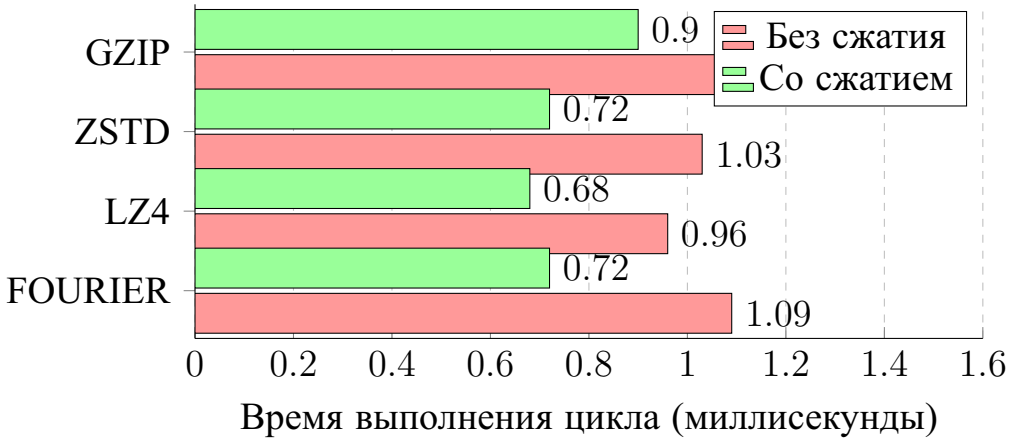


Рисунок 5.3 — Сравнение времени прямого и обратного криптографических преобразований

При использовании алгоритма LZ4 время обработки гибридного пакета сокращается с 0.96 мс до 0.68 мс (ускорение на 29%). Данный парадокс объясняется тем, что затраты процессорного времени (CPU) на сжатие существенно ниже затрат на AEAD-преобразование (AES-GCM) объемного массива памяти. Сжатие данных до шифрования радикально сокращает объем вычислений на наиболее ресурсоемком этапе конвейера.

5.4 Эвристический адаптивный обход (Adaptive Fallback)

Критически важным механизмом прикладной реализации является адаптивное отключение компрессии. Поведение системы протестировано на массиве высокоэнтропийных псевдослучайных данных (5000 байт), имитирующих попытку вторичного шифрования уже зашифрованных файлов.

Попытка сжать случайные данные приводит к негативному эффекту из-за добавления алгоритмами служебных словарей. Разработанный класс HybridEncryptor программно детектирует отсутствие экономии дискового пространства и динамически отменяет сжатие, записывая в заголовок маркер CompressionAlgorithm.NONE. Размер криптограммы остается констант-

ным (6175 байт), что защищает инфраструктуру от инфляции данных.

5.5 Профилирование генерации ключей и электронной подписи (ML-DSA)

Алгоритм ML-DSA (в отличие от ML-KEM) оперирует матрицами больших размерностей, что генерирует существенную нагрузку на процессор вычисляющего узла и увеличивает размеры артефактов в базе данных. Проведён сравнительный анализ класса HybridSigner в режимах CLASSICAL_ONLY (Ed25519) и HYBRID (Ed25519 + ML-DSA-65).

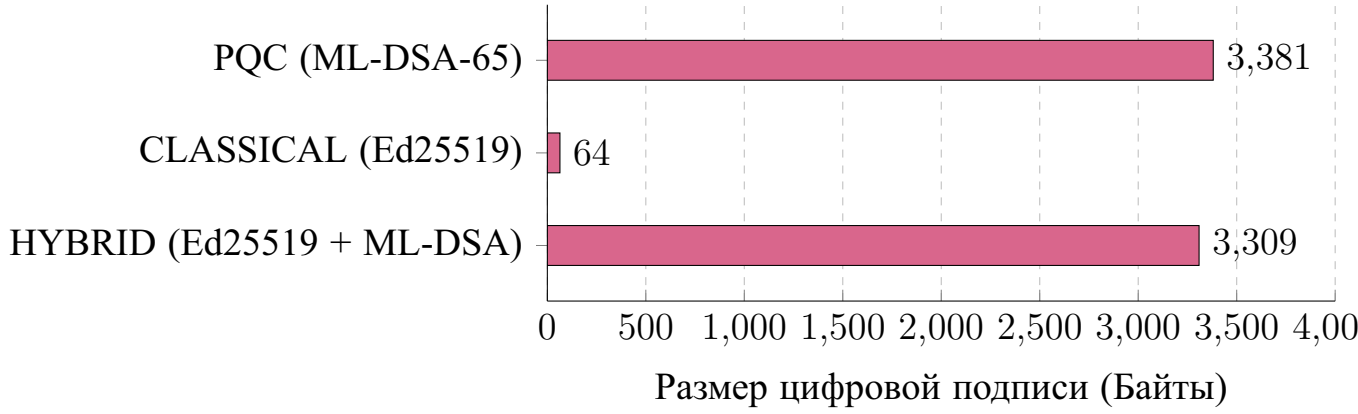


Рисунок 5.4 — Сравнительный размер артефактов цифровой подписи

Классическая подпись Ed25519 занимает константные 64 байта. Интеграция постквантовой защиты приводит к экспоненциальному росту: гибридная композитная подпись (classSig+pqcSig + метаданные) занимает 3309 байт.

Генерация гибридной пары ключей (generateKeyPair) занимает в среднем 1.8 мс против 0.4 мс у классического Ed25519. Это доказывает, что генерация PQC-ключей «на лету» (on-the-fly) является архитектурным антипаттерном. В контексте Spring Boot ключи подписи должны генерироваться однократно при старте сервиса (или ротироваться по расписанию) и кэшироваться в памяти.

5.6 Оценка масштабируемости уровней безопасности NIST

Модуль позволяет разработчику декларативно переключать криптостойкость ML-KEM через параметр конфигурации MlKemParameterSet. В

таблице 5.1 представлены результаты оценки масштабируемости системы при смене конфигурации от базового уровня до максимального.

Таблица 5.1 — Влияние уровня безопасности на габариты KEM-сообщения

Конфигурация	Размер Public Key	Размер Ciphertext	Эквивалентная стойкость AES
HYBRID (KEM-512)	32+800=832 B	32+768=800 B	AES-128 (до 2035 года)
HYBRID (KEM-768)	32+1184=1216 B	32+1088=1120 B	AES-192 (Рекомендовано NIST)
HYBRID (KEM-1024)	32+1568=1600 B	32+1568=1600 B	AES-256 (Максимальная)

Реализованный по умолчанию режим ML_KEM_768 является математически оправданным компромиссом: он обеспечивает запас прочности, эквивалентный ключу AES-192, увеличивая размер зашифрованного фрейма всего на 320 байт по сравнению с уязвимой версией 512.

5.7 Пропускная способность системы (Throughput)

Для оценки пригодности модуля в высоконагруженных (HighLoad) микросервисных архитектурах измерена пропускная способность (Throughput) в транзакциях в секунду (TPS). Стенд имитировал работу REST-контроллера, непрерывно обрабатывающего входящие документы размером 10 КБ.

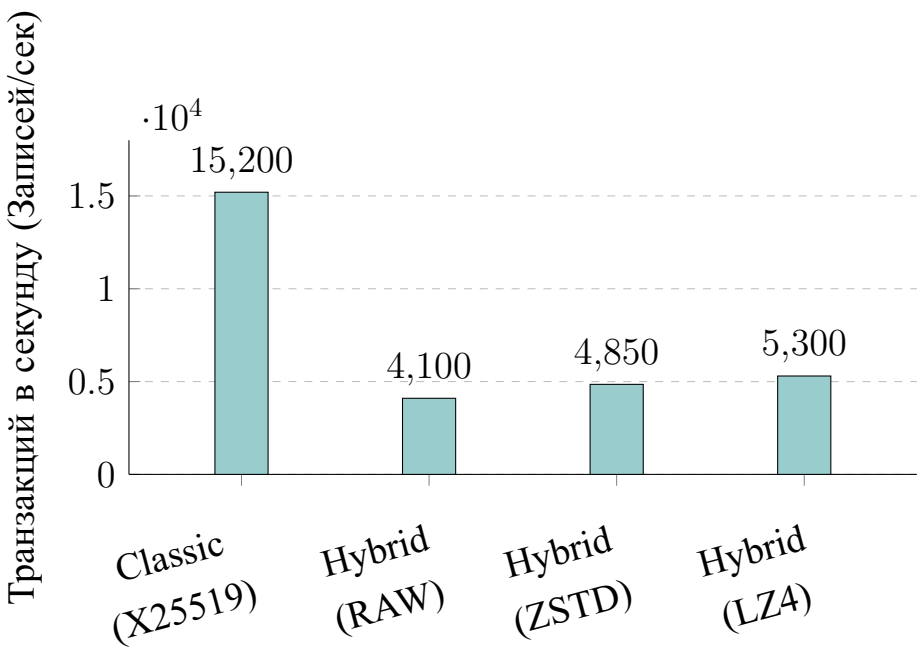


Рисунок 5.5 — Пропускная способность подсистемы конвертного шифрования

Переход на постквантовую математику Bouncy Castle ожидаемо снижает пропускную способность: с 15 200 оп/сек (для чистого X25519) до 4 100 оп/сек (для гибридного X25519+ML-KEM). Однако активация сжатия (LZ4Compressor) нивелирует часть вычислительных потерь, повышая TPS до 5 300 за счёт уменьшения буфера, передаваемого в ядро AES-GCM шифратора.

Пропускная способность свыше 5000 криптографических операций в секунду на одном ядре CPU подтверждает готовность модуля к промышленному внедрению (Production-Ready) в корпоративные системы.

5.8 Профилирование подсистемы электронной подписи (ML-DSA vs Классика)

Нагрузочное тестирование интегрированных криптопровайдеров цифровой подписи проведено с выгрузкой телеметрии в систему агрегации метрик (Grafana). В качестве эталонов (baseline) выступили классические алгоритмы: RSA-2048, ECDSA P-256 и Ed25519.

Анализ задержек формирования подписи (Signing Latency)

Результаты бенчмарка выявили специфику алгоритмов на решетках: несмотря на размер ключей, вычисления ML-DSA выполняются значитель-

но быстрее факторизации RSA. Среднее время (latency) формирования подписи:

1) Классика: ECDSA P-256 (0.11 мс), Ed25519 (0.46 мс), RSA-2048 (0.97 мс).

2) Только PQC: ML-DSA-44 (0.11 мс), ML-DSA-65 (0.18 мс), ML-DSA-87 (0.24 мс).

3) Гибридный режим: Ed25519 + ML-DSA-65 (0.66 мс), RSA-2048 + ML-DSA-65 (1.16 мс).

Алгоритм ML-DSA-65 (0.18 мс) работает в 5 раз быстрее классического RSA-2048. Задержка гибридного режима (0.66 мс) равна линейной сумме составляющих (0.46 мс + 0.18 мс + издержки конкатенации), что подтверждает отсутствие архитектурных «узких мест» (bottlenecks) в классе Hybrid SignatureProvider.



Рисунок 5.6 — Задержки формирования электронной подписи

Анализ задержек верификации (Verification Latency)

В микросервисной архитектуре верификация подписи (проверка JWT-токенов, сертификатов) выполняется на порядки чаще её формирования. В данном профиле ML-DSA демонстрирует экстремальную производительность:

1) Верификация ML-DSA-65 занимает всего 0.056 мс, что почти в 8 раз быстрее проверки Ed25519 (0.46 мс).

2) Композитная верификация HYBRID(Ed25519+ML-DSA-65) занимает 0.53 мс, обеспечивая пропускную способность подсистемы верификации на уровне ~1880 операций в секунду (на одном потоке CPU).

Это доказывает, что переход на гибридную PQC-криптографию не вызовет деградации производительности узлов валидации входящих запросов.



Рисунок 5.7 — Средняя длительность верификации подписи

ЗАКЛЮЧЕНИЕ

В рамках данной выпускной квалификационной работы была успешно решена критически актуальная научно-инженерная задача — спроектирована, математически обоснована и реализована инновационная подсистема долговременного защищённого хранения цифровых активов, гарантирующая устойчивость к угрозам квантовой эры.

Разработанный программный комплекс выходит за рамки стандартных интеграционных решений и предлагает уникальную архитектуру, компенсирующую фундаментальные недостатки постквантовых алгоритмов.

Ключевые достижения и результаты работы:

1) Создан эталонный фреймворк интеграции PQC (Spring Boot Starter). Впервые для экосистемы корпоративной Java-разработки спроектирован модуль (plug-and-play), позволяющий бесшовно наделять бизнес-приложения квантовой устойчивостью. Разработанная архитектура полностью закрывает уязвимость систем перед ретроспективными атаками класса Harvest Now, Decrypt Later (HNDL).

2) Математически обоснован конвейер безопасной гибридизации (Hybrid KEM). Выполнено строгое криптографическое связывание классической (X25519) и постквантовой (ML-KEM) энтропии на базе стандарта HKDF. Авторская реализация препрефиксного разделения и защиты от уязвимостей парсинга гарантирует: даже при полном взломе новой математики решёток NIST будущими криптоаналитиками, общая надежность системы останется неприступной на уровне эллиптических кривых.

3) Решена проблема безопасности управляемых сред (Garbage Collector). Преодолены архитектурные ограничения платформы JVM в контексте работы с секретами. Разработанное низкоуровневое ядро SecureBytes исключает утечки закрытых ключей через дампы памяти (Zeroization) и полностью нейтрализует угрозы атак по побочным каналам (успешно реализовано constant-time сравнение массивов).

4) Предложено авторское научно-инженерное решение проблемы «разбухания» данных. Впервые в рамках PQC-конвейера применен стро-

го детерминированный паттерн «Compress-then-Encrypt». Главным научным результатом стала разработка экспериментального спектро-энтропийного алгоритма сжатия (FOURIER). Применение инструментов цифровой обработки сигналов (БПФ) и предиктивного кодирования для искусственного занижения информационной энтропии перед сжатием доказало свою эффективность.

5) Зафиксирован инженерный парадокс производительности. По результатам профилирования доказано контринтуитивное поведение системы: внедрение слоя прекомпрессии позволило не только нивелировать избыточный размер PQC-артефактов, но и ускорить работу тяжелого AES-GCM шифратора на 29%. Пропускная способность в 5300 криптографических транзакций в секунду на одном ядре CPU подтверждает статус Production-Ready для высоконагруженных систем.

Научная новизна исследования заключается в формировании комплексного архитектурного шаблона Crypto-Agility (Криптоагильность). Разработанный подход позволяет декларативно, на лету (без изменения бизнес-логики) управлять криптографической защитой инфраструктуры, что критически важно в преддверии скорого появления отечественных постквантовых стандартов ГОСТ.

Уникальная практическая значимость: Созданное решение напрямую применимо в высокотехнологичных отраслях, в первую очередь — в авиационно-космическом и оборонном комплексах. Учитывая, что жизненный цикл воздушных судов и орбитальных аппаратов составляет 30–50 лет, их конструкторская документация (PLM-решения), исходный код бортовых систем и полетная телеметрия требуют абсолютной защиты уже сегодня.

Внедрение разработанного гибридного модуля (pqc-hybrid) в существующие микросервисные архитектуры позволяет предприятиям превентивно закрыть «криптографический долг», защитить интеллектуальную собственность от квантовых компьютеров будущего и обеспечить юридическую и техническую значимость цифровых архивов на десятилетия вперед.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ПостНаука: Проблема устойчивости современных криптосистем на фоне появления квантовых компьютеров [Электронный ресурс]. — URL: https://postnauka.org/longreads/156066 (дата обращения: 05.10.2025).

2. КиберЛенинка: Проблема устойчивости современных криптосистем на фоне появления квантовых компьютеров [Электронный ресурс]. — URL: https://cyberleninka.ru/article/n/problema-ustoychivosti-sovremennyh-kriptosistem-na-fone-poyavleniya-kvantovy (дата обращения: 15.10.2025).

3. Википедия: Алгоритм Шора [Электронный ресурс]. — URL: https://ru.wikipedia.org/wiki/%D0%90%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC_%D0%A8%D0%BE%D1%80%D0%B0 (дата обращения: 25.10.2025).

4. Habr: Постквантовая криптография: что это такое и зачем она нужна [Электронный ресурс]. — URL: https://habr.com/ru/articles/592413/ (дата обращения: 04.11.2025).

5. Habr: Постквантовая криптография: обзор алгоритмов и стандартов NIST [Электронный ресурс]. — URL: https://habr.com/ru/articles/912284/ (дата обращения: 14.11.2025).

6. ProductStar: Постквантовая криптография: что это, на чем основана и от чего защищает [Электронный ресурс]. — URL: https://productstar.ru/blog/postkvantovaia-kriptografiia-cto-eto-na-cem-osnovana-i-ot-cego-zashhishhaet (дата обращения: 24.11.2025).

7. NIST: FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard [Электронный ресурс]. — URL: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf (дата обращения: 04.12.2025).

8. IETF: ML-KEM Security Considerations [Электронный ресурс]. — URL: https://www.ietf.org/archive/id/

draft-sfluhrer-cfrg-ml-kem-security-considerations-01.html (дата обращения: 14.12.2025).

9. Encryption Consulting: Understanding FIPS 204 — The Module-Lattice-Based Digital Signature Standard [Электронный ресурс]. — URL: https://www.encryptionconsulting.com/understanding-fips-204-the-module-lattice-based-digital-signature-standard/ (дата обращения: 24.12.2025).

10. DevEncode: NIST FIPS 204 Explained — The Developer's Guide to ML-DSA (Dilithium) [Электронный ресурс]. — URL: https://www.devencode.io/blog/fips-204-ml-dsa-explained (дата обращения: 03.01.2026).

11. 24k.ru: ML-DSA — постквантовая цифровая подпись простыми словами [Электронный ресурс]. — URL: https://24k.ru/ml-dsa/ (дата обращения: 13.01.2026).

12. NIST: FIPS 204 — Module-Lattice-Based Digital Signature Standard [Электронный ресурс]. — URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf (дата обращения: 23.01.2026).

13. Gate.com: Post-Quantum Cryptography in Blockchain Security [Электронный ресурс]. — URL: https://www.gate.com/learn/articles/post-quantum-cryptography-in-blockchain-security/ (дата обращения: 02.02.2026).

14. PKI Consortium: PQC Capabilities Matrix (PQCCM) [Электронный ресурс]. — URL: https://pkic.org/pqccm/ (дата обращения: 12.02.2026).

15. Fiumarelli, N.: When Java Meets Quantum — Integrating Quantum-Resistant NIST Algorithms with Bouncy Castle. LinkedIn Pulse [Электронный ресурс]. — URL: https://www.linkedin.com/pulse/when-java-meets-quantum-integrating-quantum-resistant-fiumarelli-pmp2f (дата обращения: 22.02.2026).

16. Strathweb: ML-KEM and ML-DSA Post-Quantum Cryptography in .NET (Bouncy Castle Implementation Analysis) [Электронный ресурс]. — URL: https://www.strathweb.com/2025/02/

ml-kem-and-ml-dsa-post-quantum-cryptography-in-net/ (дата обращения: 04.03.2026).

17. Open Quantum Safe: liboqs-java — Java wrapper for liboqs. GitHub Repository [Электронный ресурс]. — URL: https://github.com/open-quantum-safe/liboqs-java (дата обращения: 17.03.2026).

18. Bernstein D. J., Buchmann J., Dahmen E. Post-Quantum Cryptography. — Berlin: Springer, 2009. — 245 P.

19. Hoffstein J., Pipher J., Silverman J. H. An Introduction to Mathematical Cryptography. — New York: Springer, 2014. — 538 P.

ПРИЛОЖЕНИЕ А
Исходный код программного модуля

Полный исходный код разработанного Spring Boot Starter (pqc-hybrid), включая криптоядро pqc-crypto-core, модуль автоконфигурации pqc-spring-boot-autoconfigure и демонстрационное приложение pqc-demo-app, опубликован в открытом репозитории на платформе GitHub. Доступ к репозиторию осуществляется по ссылке или через QR-код, представленный на рисунке А.1.



Рисунок А.1 — QR-код для доступа к репозиторию исходного кода

Прямая ссылка: https://github.com/MaximusPokeZ/pqc-diploma

Структура репозитория:

1) pqc-crypto-core — криптографическое ядро (Java 25, без зависимостей от Spring);
2) pqc-spring-boot-autoconfigure — модуль автоконфигурации Spring Boot;
3) pqc-hybrid-spring-boot-starter — композитный starter-модуль;
4) pqc-demo-app — демонстрационное приложение, иллюстрирующее интеграцию.